

Memory Hierarchy & Performance Analysis

Answers to Assignment Questions: Cache, DRAM, MRAM, NAND Flash, RRAM, Latency, Bandwidth, and Virtual Memory Paging

Prepared for: Case-study questions on smartphones, cloud servers, autonomous vehicles, edge AI devices, gaming consoles, and the five application scenarios below.

Question 1: High-Frequency Trading System

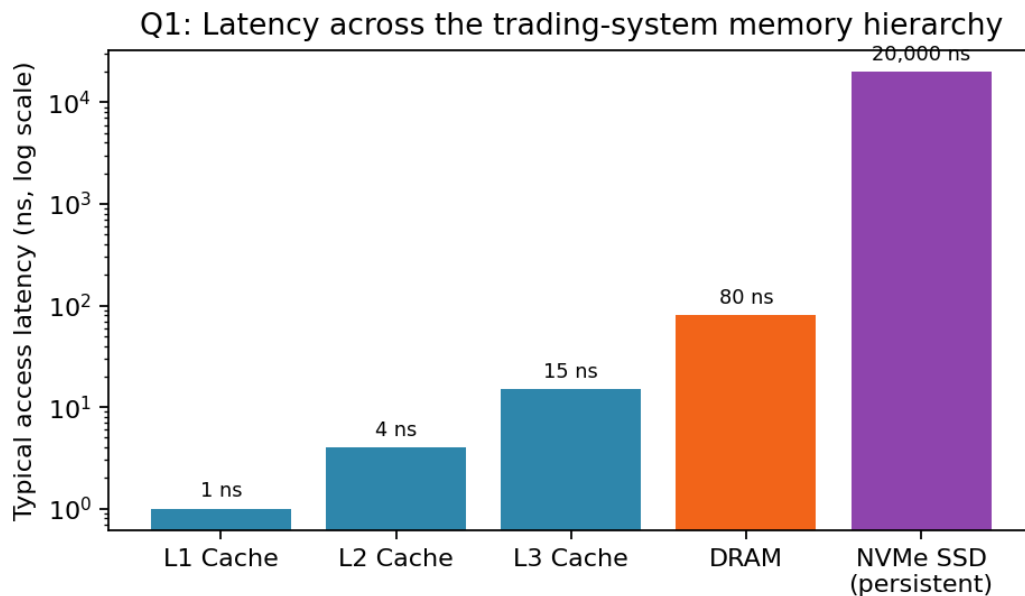
Concept overview

A trading system re-reads the same order-book entries and price ticks thousands of times a second, so it lives or dies on **latency**, not capacity. The memory hierarchy has three layers to consider: **CPU caches** (L1/L2/L3 SRAM) for the hot order-book data, **DRAM** for the working set of market-data streams, and **persistent storage** (NVMe SSD/Optane) for trade logs and compliance records that must survive a crash.

Role of each layer

- **CPU caches (L1/L2/L3)**: hold the active order book, price ladders, and hot trading-strategy variables. Nanosecond access lets the CPU react to a price tick within one clock cycle range.
- **DRAM**: buffers incoming market-data feeds (tens of GB/s bandwidth) and holds the full instrument universe that does not fit in cache.
- **Persistent storage (NVMe SSD)**: asynchronously logs every trade for audit/replay; kept off the critical path so its higher latency never blocks a trade decision.

Latency comparison



Performance computation (Average Memory Access Time, AMAT)

AMAT = Hit time + (Miss rate × Miss penalty). Example: if L1 hit time = 1 ns, L1 miss rate = 5%, and the penalty of going to L2 (15 ns) applies on a miss:

AMAT = 1 + (0.05 × 15) = **1.75 ns** — about 43× faster than fetching directly from DRAM (80 ns) and roughly 11,000× faster than going to the NVMe SSD (20,000 ns) on every access. This is why keeping the order book cache-resident is the single biggest lever for throughput.

Proposed hierarchy

Layer	Holds	Optimization technique
L1/L2 cache (per core)	Active order book, hot variables	Data locality: keep instrument data structures small & aligned
L3 shared cache	Cross-core shared price feeds	Cache partitioning to avoid strategy threads evicting each other
DRAM	Full instrument set, feed buffers	High-bandwidth, multi-channel DRAM; huge pages to cut TLB misses
NVMe SSD / Optane	Trade logs, compliance records	Asynchronous write-behind logging, off the hot path

Interpretation

The dominant bottleneck in such systems is a cache miss that falls through to DRAM or, worse, storage.

Keeping the working set small and cache-friendly (data locality) and prefetching upcoming ticks are far more effective than adding raw DRAM capacity. This hierarchy minimizes access delay for the hot path while still guaranteeing durability for compliance data.

Question 2: Medical Imaging Workstation

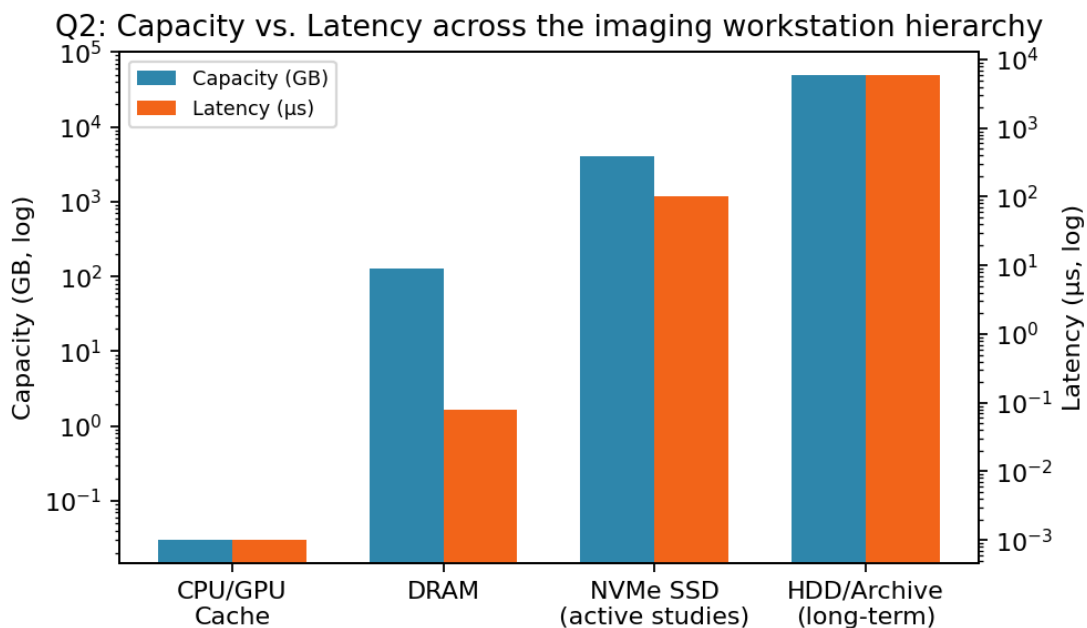
Concept overview

CT/MRI datasets are large (hundreds of MB to several GB per study) but only small regions are actively viewed or filtered at any moment. The workload therefore needs a hierarchy that balances **capacity** (to hold whole studies) with **bandwidth** (to stream voxel data to the GPU/CPU for enhancement and rendering).

Interaction of the layers

- **Cache (CPU/GPU L1-L3):** holds the image tile or slice currently being filtered/rendered — keeps convolution kernels and small windows cache-resident for real-time enhancement.
- **DRAM:** holds the full 3-D volume of the study in view, enabling fast scrolling through slices without re-reading storage; also buffers multiple concurrently open studies.
- **High-capacity storage (NVMe SSD for active cases, HDD/PACS archive for older studies):** provides the multi-TB capacity needed for a hospital's imaging archive at a much lower cost per GB.

Capacity vs. latency trade-off



Performance measures

Throughput for opening a study $\approx$ dataset size $\div$ effective bandwidth. A 1 GB CT volume read from an NVMe SSD at ~ 3 GB/s loads in roughly 0.33 s, versus several seconds from a network HDD archive at ~ 150 MB/s. Once loaded into DRAM (bandwidth in tens of GB/s) and cached in SRAM for the active slice, subsequent scroll/zoom operations complete in well under a millisecond.

Proposed hierarchy

Layer	Role	Technique applied
GPU/CPU cache	Active slice / filter kernel	Tiling for data locality during convolution

Layer	Role	Technique applied
DRAM (large capacity)	Whole study volume in view	Prefetch adjacent slices while scrolling
NVMe SSD (hot tier)	Recently/actively opened studies	SSD caching layer in front of archive
HDD/PACS archive (cold tier)	Long-term study retention	Hierarchical storage management, compression

Interpretation

An SSD hot-tier in front of the archive removes the biggest bottleneck (slow cold storage) at modest cost, while DRAM sizing determines how many studies can be held open simultaneously. This tiered design balances latency, capacity, and cost far better than an all-flash or all-cache approach would.

Question 3: Smart Surveillance System

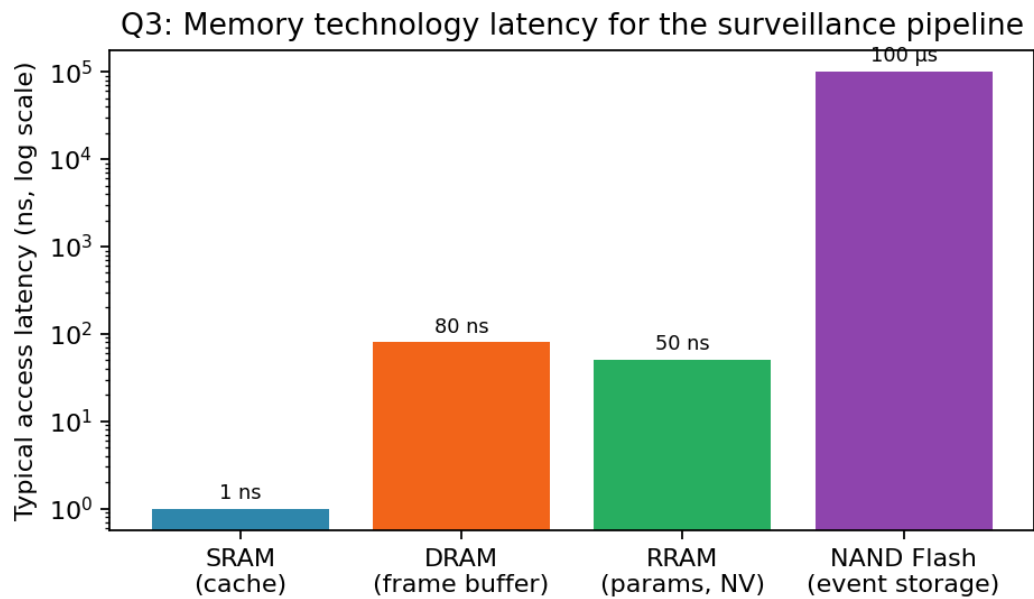
Concept overview

Real-time object detection over multiple high-resolution camera streams must sustain high **throughput** (many frames/sec across many streams) while keeping **latency** low enough for detection to be actionable, and must not lose captured video if power or network drops — favoring non-volatile storage for that role.

Memory requirements by data type

- **Video frame buffers:** large, continuously overwritten — best placed in DRAM for its bandwidth and capacity; only the current frame's active region touches cache.
- **Intermediate CNN activations:** reused heavily within a single inference pass — kept in SRAM cache for nanosecond access, since they are accessed many times per frame.
- **Detection model weights / parameters:** read-mostly and frequently re-used across frames — candidates for fast non-volatile memory (RRAM) so they persist without refresh power.
- **Recorded event clips / alerts:** must survive power loss — written to NAND flash.

Technology comparison



Technology	Volatile?	Typical role here	Key trade-off
SRAM (cache)	Yes	CNN intermediate activations	Fastest, smallest, most expensive/bit
DRAM	Yes	Multi-camera frame buffers	High bandwidth, needs refresh power
RRAM	No	Model weights, config	Fast non-volatile write, moderate endurance
NAND Flash	No	Recorded clips, event logs	High capacity/low cost, slowest of the four

Performance measure example

For 4 cameras at 1080p/30fps, raw bandwidth demand $\approx 4 \times (1920 \times 1080 \times 1.5 \text{ bytes} \times 30) \approx \textbf{\sim 370 MB/s}$ just for frame ingestion — comfortably inside DRAM bandwidth (tens of GB/s) but far above what NAND flash could

sustain directly, confirming frames must land in DRAM first and only be flushed to flash as compressed, lower-rate event clips.

Interpretation

The main bottleneck risk is writing raw, uncompressed video straight to flash, which would throttle the whole pipeline. Buffering in DRAM, caching hot activations in SRAM, and storing weights in RRAM for instant-on operation gives high sustained throughput while flash is reserved for compressed, lower-bandwidth archival writes.

Question 4: Scientific Computing System (Large-Scale Simulations)

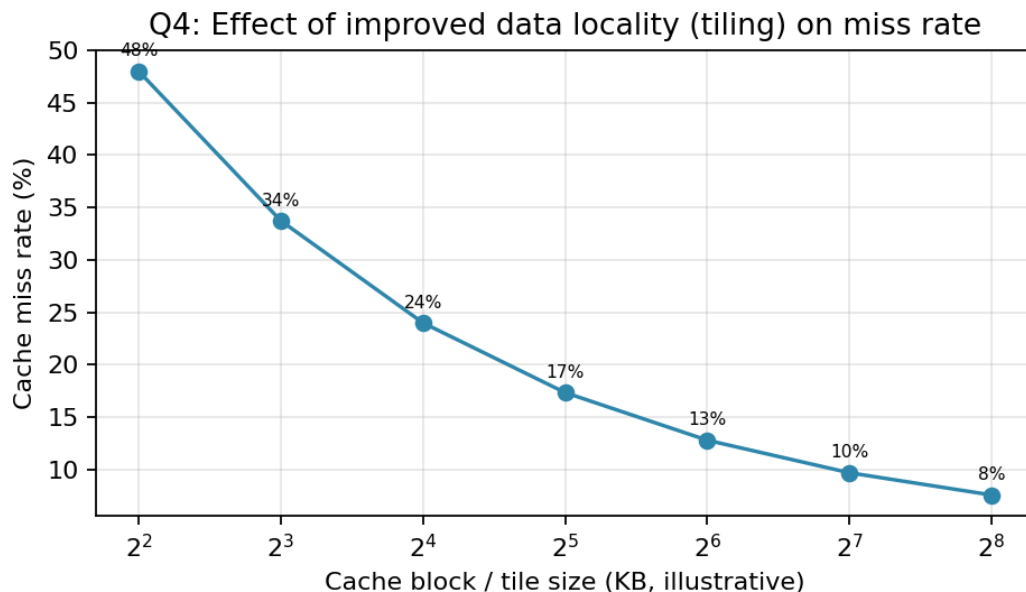
Concept overview

Matrix-heavy simulation kernels (e.g., dense linear algebra, stencil codes) repeatedly touch large arrays that do not fit in cache. Performance is governed by how well the access pattern exhibits **data locality** — both temporal (reusing the same data soon) and spatial (using nearby data) — because that determines the cache-miss rate, which in turn determines how often the slower DRAM path is used.

How each factor affects execution

- **Cache capacity:** limits how large a working set (e.g., a matrix tile) can be reused before eviction; too small a cache forces repeated DRAM trips for the same data.
- **Data locality:** naive row/column-major mismatches or huge stride accesses destroy spatial locality and inflate the miss rate even when the cache is large enough.
- **Cache misses:** each miss costs the DRAM miss penalty (tens of ns) instead of ~1 ns — with millions of accesses, even a few extra percentage points of miss rate is very costly.
- **DRAM bandwidth:** once data streams from DRAM, achieved bandwidth caps how fast the kernel can be fed, especially for streaming/stencil kernels with low reuse.
- **Memory latency:** on the critical path whenever a miss cannot be hidden by prefetching or out-of-order execution, directly stalling the compute pipeline.

Effect of improving locality (loop tiling / blocking)



Illustrative trend: as the working tile is sized to fit the cache (blocking/tiling), the miss rate drops sharply, then flattens once the tile already fits comfortably — showing why tile size should match cache capacity rather than being maximized blindly.

Performance computation

Speedup from reducing miss rate: if $AMAT_{before} = 1 + 0.30 \times 80 = 25$ ns and, after tiling, $AMAT_{after} = 1 + 0.06 \times 80 = 5.8$ ns, then **Speedup** $\approx 25 / 5.8 \approx 4.3\times$ on memory-bound sections of the kernel — even though not a single instruction of arithmetic changed.

Recommended improvements

Technique	Effect
Loop tiling / blocking	Matches working-set size to cache capacity, cutting miss rate
Data layout transformation (e.g., row-major access)	Improves spatial locality, fewer cache lines fetched
Prefetching	Hides remaining DRAM latency behind computation
Multi-channel / high-bandwidth DRAM	Raises the ceiling for streaming-dominated kernels
Larger shared L3 / NUMA-aware placement	Reduces cross-socket misses in multi-node simulations

Interpretation

For compute kernels like this, the memory hierarchy — not raw CPU clock speed — is usually the bottleneck. Restructuring the algorithm for locality is typically far more effective than buying faster DRAM, because it attacks the miss rate directly rather than only the miss penalty.

Question 5: Wearable Health-Monitoring Device

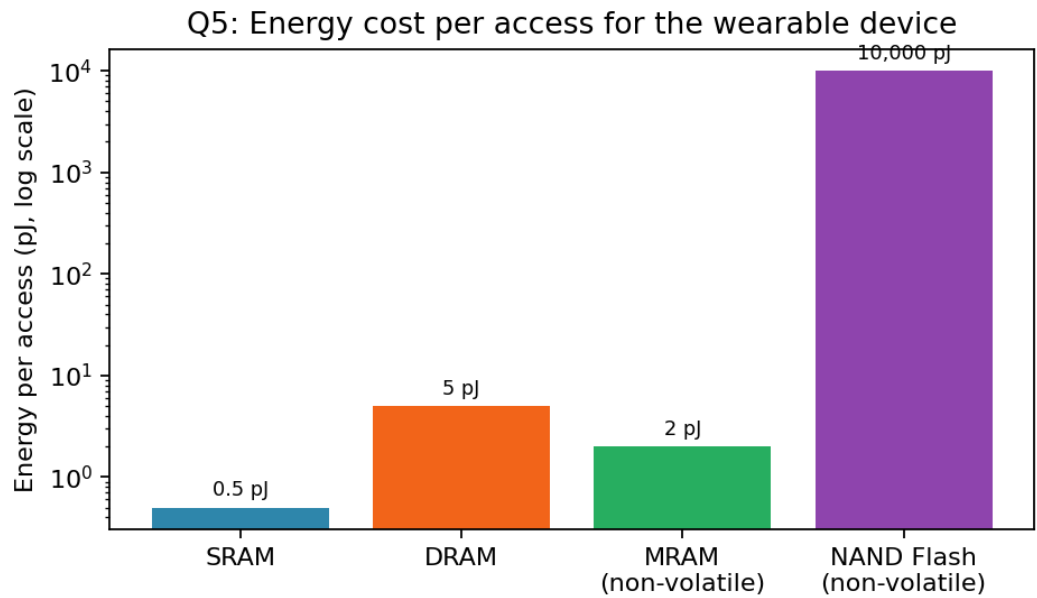
Concept overview

A wearable continuously samples sensors (heart rate, motion, SpO2) on a tiny battery, so **energy per access** and **data retention without power** matter as much as speed. The design must balance SRAM's speed, DRAM's bandwidth/capacity, and non-volatile memory's ability to retain data with zero standby power.

Trade-offs

Memory	Latency	Energy/access	Retention	Best use here
SRAM	Sub-ns – few ns	Very low, but leaks power when idle (needs constant supply)	Volatiles – loses data if power off	Acts as a buffer for the sample currently being processed
DRAM	~tens of ns	Low per access, but refresh drains battery continuously	Volatiles – loses data if power off	Avoided or minimized; only if bulk buffering
Non-volatile (MRAM/RRAM/Flash)	~ns – μ s, R/W slower	MRAM/RRAM lower, Flash higher per access	Retains data with power off	Logging history & settings between charges

Energy comparison



Why DRAM is minimized

DRAM needs periodic refresh (typically every ~64 ms) to retain data, which burns power even when the device is idle — a poor fit for a battery that must last days. MRAM/RRAM, by contrast, hold their state with no refresh at all, so the device can power down completely between sensor samples and 'wake up' instantly with no data loss.

Proposed hierarchy

Layer	Holds	Why
Small SRAM buffer	Sample currently being filtered/processed	Fastest access for the active computation, kept minimal to save power
MRAM/RRAM (main non-volatile storage)	Recent sensor history, calibration data, settings	No refresh power, instant-on, good endurance for frequent writes

Layer	Holds	Why
NAND Flash (optional, bulk)	Long-term logs synced periodically to phone	Lowest cost per bit for data not needed instantly

Interpretation

By replacing most of DRAM's role with MRAM/RRAM, the device avoids continuous refresh energy and can duty-cycle (sleep/wake) aggressively, which is the single biggest lever for battery life in always-on sensing wearables — at the cost of somewhat higher per-write energy than SRAM/DRAM, which is an acceptable trade given the low sampling rate.